Sí. Para el **Diseño A** haría el proceso completo en este orden:

$$2023 observable population \rightarrow eligible at - risk months \rightarrow 500,000 unique parties \rightarrow 1 random eligible month$$

La ventaja de este diseño es que **no condicionamos la selección por el label**. El `label_3m` se calcula únicamente después de asignar el `observation_date`.

Te recomiendo usar un hash fijo en lugar de `RAND()` porque hace que la selección sea reproducible.

```
design_a_500k_2023 <- sdf_sql(
  sc,
  "
  WITH observable_months AS (

    /* ========================================================
       STEP 1
       All party-months actually observable during 2023
       ======================================================== */

    SELECT DISTINCT
        party_id,
        data_date AS observation_date

    FROM bc3.person_restricted

    WHERE data_date >= '2023-01-01'
      AND data_date <= '2023-12-31'
      AND data_date = LAST_DAY(data_date)
  ),

  with_first_alert AS (

    /* ========================================================
       Attach first qualifying alert date
       ======================================================== */

    SELECT
        p.party_id,
        p.observation_date,
        a.first_qualifying_alert_date

    FROM observable_months p

    LEFT JOIN bu_frd.party_first_valid_alert_all a
        ON p.party_id = a.party_id
```

```
),

eligible_months AS (

    /* ============================================================
       STEP 2
       Keep only months where the party was still at risk
       of its FIRST qualifying alert
       ============================================================ */

    SELECT
        party_id,
        observation_date,
        first_qualifying_alert_date

    FROM with_first_alert

    WHERE
        first_qualifying_alert_date IS NULL
        OR first_qualifying_alert_date > observation_date
),

eligible_parties AS (

    /* ============================================================
       Parties having at least one valid observation month in 2023
       ============================================================ */

    SELECT DISTINCT
        party_id

    FROM eligible_months
),

sampled_parties AS (

    /* ============================================================
       STEP 3
       Select exactly 500,000 unique parties.

       xxhash64 gives us deterministic/reproducible
       pseudo-random ordering.
       ============================================================ */

    SELECT
        party_id

    FROM (
```

```
        SELECT
            party_id,

            ROW_NUMBER() OVER (
                ORDER BY
                    XXHASH64(
                        CAST(party_id AS STRING),
                        'VULNEX_2023_SAMPLE_V1'
                    )
            ) AS sample_rank

        FROM eligible_parties

    ) x

    WHERE sample_rank <= 500000
),

sampled_eligible_months AS (

    /* ============================================================
       Recover all eligible 2023 months for the selected parties
       ============================================================ */

    SELECT
        e.party_id,
        e.observation_date,
        e.first_qualifying_alert_date

    FROM eligible_months e

    INNER JOIN sampled_parties s
        ON e.party_id = s.party_id
),

randomized_months AS (

    /* ============================================================
       STEP 4
       Randomly rank the ELIGIBLE months separately
       for every sampled party
       ============================================================ */

    SELECT
        party_id,
        observation_date,
        first_qualifying_alert_date,
```

```
        ROW_NUMBER() OVER (
            PARTITION BY party_id

            ORDER BY
                XXHASH64(
                    CAST(party_id AS STRING),
                    CAST(observation_date AS STRING),
                    'VULNEX_2023_MONTH_V1'
                )
        ) AS random_month_rank

    FROM sampled_eligible_months
),

assigned_observation AS (

    /* ============================================================
       Keep exactly one random eligible month per party
       ============================================================ */

    SELECT
        party_id,
        observation_date,
        first_qualifying_alert_date

    FROM randomized_months

    WHERE random_month_rank = 1
),

final_labeled AS (

    /* ============================================================
       STEP 5
       Create the prospective 3-month outcome
       ============================================================ */

    SELECT
        party_id,

        observation_date,

        YEAR(observation_date)
            AS observation_year,

        MONTH(observation_date)
            AS observation_month,
```

```
        first_qualifying_alert_date,

        CASE

            WHEN first_qualifying_alert_date > observation_date

             AND first_qualifying_alert_date
                <= ADD_MONTHS(observation_date, 3)

            THEN 1

            ELSE 0

        END AS label_3m

    FROM assigned_observation
 )

 SELECT *
 FROM final_labeled

 ORDER BY
    observation_date,
    party_id
 "
)

design_a_500k_2023
```

1. Qué debe producir

La tabla final tendrá exactamente una fila por party:

party_id	observation_date	first_qualifying_alert_date	label_3m
A	2023-03-31	NULL	0
B	2023-07-31	2023-09-15	1
C	2023-11-30	2024-04-10	0
...	...	...	...

Por tanto deberíamos tener:

$$\boxed{500{,}000\ \text{rows}}$$

y:

$$\boxed{500{,}000\ \text{unique party_id}}$$

---

2. Resultado general de la prueba

Después corre:

```
design_a_summary <- design_a_500k_2023 %>%
  summarise(
    total_parties = n(),
    unique_parties = n_distinct(party_id),

    label_1 = sum(label_3m),

    label_0 = n() - sum(label_3m),

    label_1_rate = mean(label_3m),

    label_1_rate_pct = 100 * mean(label_3m)
  ) %>%
  collect()

design_a_summary
```

Aquí la tasa será:

$$\boxed{ Rate= \frac{N(label_3m=1)} {500{,}000} }$$

Esta tasa **no está afectada por el sampling 10:1**, porque no seleccionamos parties basándonos en su label.

---

3. Resultado por observation month

Este es probablemente el resultado más útil:

```
design_a_monthly <- design_a_500k_2023 %>%
  group_by(
    observation_year,
    observation_month,
    observation_date
  ) %>%
  summarise(
    assigned_parties = n(),
```

```
  label_1 = sum(label_3m),

  label_0 = n() - sum(label_3m),

  label_1_rate = mean(label_3m),

  label_1_rate_pct = 100 * mean(label_3m)
) %>%
arrange(observation_date) %>%
collect()
```

design_a_monthly

El resultado será aproximadamente:

Observation Month	Assigned Parties	Label 1	Label 0	Label 1 Rate
Jan-2023	...	...	...	...%
Feb-2023	...	...	...	...%
Mar-2023	...	...	...	...%
...	...	...	...	...%
Dec-2023	...	...	...	...%

No necesariamente tendrás exactamente:

$$500,000/12 = 41,667$$

en cada mes.

Eso es esperado en el Diseño A porque cada party solo puede ser asignado entre **sus meses realmente elegibles**.

---

4. Cuántos meses elegibles tenía cada party

Te recomiendo guardar también esta información porque ayuda a interpretar el random assignment:

```
eligible_months_distribution <- sdf_sql(
  sc,
  "
  WITH population AS (
```

```
    SELECT DISTINCT
        party_id,
        data_date AS observation_date

    FROM bc3.person_restricted

    WHERE data_date >= '2023-01-01'
      AND data_date <= '2023-12-31'
      AND data_date = LAST_DAY(data_date)
),

eligible AS (

    SELECT
        p.party_id,
        p.observation_date

    FROM population p

    LEFT JOIN bu_frd.party_first_valid_alert_all a
        ON p.party_id = a.party_id

    WHERE
        a.first_qualifying_alert_date IS NULL
        OR a.first_qualifying_alert_date > p.observation_date
),

party_coverage AS (

    SELECT
        party_id,
        COUNT(*) AS eligible_months_2023

    FROM eligible

    GROUP BY party_id
)

SELECT
    eligible_months_2023,

    COUNT(*) AS parties

FROM party_coverage

GROUP BY eligible_months_2023
```

```
  ORDER BY eligible_months_2023
  "
) %>%
  collect()
```

eligible_months_distribution

Así sabremos cuántos parties tenían:

1 eligible month
2 eligible months
...
12 eligible months

Esto no es para cambiar la prueba; sirve para entender la población sobre la que estamos randomizando.

Un ejemplo de por qué el query funciona

Supongamos:

Party A
First qualifying alert = 2023-09-15

y el party aparece de enero a diciembre.

Los meses posteriores al evento quedan fuera:

Jan ✓
Feb ✓
Mar ✓
Apr ✓
May ✓
Jun ✓
Jul ✓
Aug ✓

Sep ✗
Oct ✗
Nov ✗
Dec ✗

De esos meses válidos seleccionamos aleatoriamente uno.

Si seleccionamos:

2023-07-31

entonces:

$$31 - Jul < 15 - Sep \leq 31 - Oct$$

y:

$$label_3m = 1$$

Pero si seleccionamos:

2023-02-28

septiembre está fuera de los próximos tres meses y:

$$label_3m = 0$$

Ese comportamiento es exactamente lo que queremos medir.

Una precisión metodológica

Este experimento construye una **muestra aleatoria prospectiva de 500,000 parties de la población observable/at-risk de 2023**. No es la misma cohorte de 688,589 utilizada por el modelo, y tampoco pretende reproducir el sampling 10:1.

Su objetivo es estimar algo mucho más natural:

> **Si seleccionamos parties independientemente del outcome y les asignamos una fecha de observación válida, ¿qué proporción presenta un first qualifying alert dentro de los siguientes 3 meses?**

Para la prueba que te solicitaron, este Diseño A es adecuado.